

SAEPestoque

Sistema de Controle de Estoque e Produtos

Aplicacao Spring Boot + MySQL + SPA (HTML/CSS/JS) para controle de inventario, produtos, fornecedores e movimentacoes.

Manual de Construcao — Versao 1.0

Stack: Spring Boot 3.2.3 + MySQL 8 + HTML/CSS/JS (SPA)

Etapa 0 — Modelagem do Banco de Dados

0.1 Diagrama Conceitual

A aplicacao **SAPEstoque** possui 4 tabelas principais: `tb_funcionarios` , `tb_produtos` , `tb_etiquetas` e `tb_anotacoes` .

Tabela	Descricao	Relacionamentos
<code>tb_funcionarios</code>	Cadastro de funcionarios do sistema	1 : N com <code>tb_produtos</code> , <code>tb_etiquetas</code> , <code>tb_anotacoes</code>
<code>tb_produtos</code>	Registros principais do dominio	N : 1 com <code>tb_funcionarios</code> , 1 : N com <code>tb_etiquetas</code> e <code>tb_anotacoes</code>
<code>tb_etiquetas</code>	Reacoes/interacoes dos funcionarios	N : 1 com <code>tb_funcionarios</code> e <code>tb_produtos</code>
<code>tb_anotacoes</code>	Comentarios/observacoes	N : 1 com <code>tb_funcionarios</code> e <code>tb_produtos</code>

Etapa 1 — Setup do Projeto Spring Boot + MySQL

1.1 Criar projeto via Spring Initializr

Dependências: Spring Web, Spring Data JPA, MySQL Driver, Lombok.

1.2 Configurar `pom.xml`

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.3</version>
</parent>
<dependencies>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId></dependency>
  <dependency><groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId><scope>runtime</scope></dependency>
  <dependency><groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId><optional>true</optional></dependency>
</dependencies>
```

1.3 Configurar o MySQL — `application.properties`

```
spring.application.name=saepestoque
spring.datasource.url=jdbc:mysql://localhost:3306/saepestoque?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=
true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=037397
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.sql.init.mode=always
spring.jpa.defer-datasource-initialization=true
```

Checkpoint Etapa 1: A aplicação sobe sem erros em `http://localhost:8080` e o banco `saepestoque` é criado.

Etapa 2 — Criar as Entidades (Model)

2.1 Funcionario.java

```
package com.saep.saepestoque.model;
import jakarta.persistence.*;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.time.LocalDateTime;

@Data @NoArgsConstructor @Entity
@Table(name = "tb_funcionarios")
public class Funcionario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private String email;
    private String nomeUsuario;
    private String imagem;
    private String senha;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}
```

2.2 Produto.java

```
package com.saep.saepestoque.model;
import jakarta.persistence.*;
import lombok.Data; import lombok.NoArgsConstructor;
import java.time.LocalDateTime; import java.util.List;

@Data @NoArgsConstructor @Entity
@Table(name = "tb_produtos")
public class Produto {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String titulo;
    private String autor;
    private String genero;
    private Integer anoPublicacao;
    private Integer numeroPaginas;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    @ManyToOne @JoinColumn(name = "funcionario_id")
    private Funcionario funcionario;
    @OneToMany(mappedBy = "produto", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Etiqueta> etiquetas;
    @OneToMany(mappedBy = "produto", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Anotacao> anotacoes;
}
```

2.3 Etiqueta.java

```
package com.saep.saepestoque.model;
import jakarta.persistence.*;
import lombok.Data; import lombok.NoArgsConstructor;
import com.fasterxml.jackson.annotation.JsonIgnore;

@Data @NoArgsConstructor @Entity
@Table(name = "tb_etiquetas")
public class Etiqueta {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @ManyToOne @JoinColumn(name = "funcionario_id")
    private Funcionario funcionario;
```

```

    @ManyToOne @JoinColumn(name = "produto_id")
    @JsonIgnore // evita loop infinito na serializacao JSON
    private Produto produto;
}

```

2.4 Anotacao.java

```

package com.saep.saepestoque.model;
import jakarta.persistence.*;
import lombok.Data; import lombok.NoArgsConstructor;
import com.fasterxml.jackson.annotation.JsonIgnore;

@Data @NoArgsConstructor @Entity
@Table(name = "tb_anotacoes")
public class Anotacao {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String texto;
    @ManyToOne @JoinColumn(name = "funcionario_id")
    private Funcionario funcionario;
    @ManyToOne @JoinColumn(name = "produto_id")
    @JsonIgnore
    private Produto produto;
}

```

Checkpoint Etapa 2: Ao subir a aplicacao, as 4 tabelas sao criadas no MySQL (verifique com `SHOW TABLES;`).

Etapa 3 — Criar os Repositories

3.1 FuncionarioRepository.java

```
package com.saep.saepestoque.repository;
import com.saep.saepestoque.model.Funcionario;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface FuncionarioRepository extends JpaRepository<Funcionario, Long> {
    Optional<Funcionario> findByEmailAndSenha(String email, String senha);
}
```

3.2 ProdutoRepository.java

```
package com.saep.saepestoque.repository;
import com.saep.saepestoque.model.Produto;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface ProdutoRepository extends JpaRepository<Produto, Long> {
    @Query("SELECT a FROM Produto a WHERE :tipo IS NULL OR a.genero = :tipo ORDER BY a.createdAt DESC")
    Page<Produto> findByGenero(@Param("tipo") String tipo, Pageable pageable);

    long countByFuncionarioId(Long funcionarioId);

    @Query("SELECT COALESCE(SUM(a.numeroPaginas), 0) FROM Produto a WHERE a.funcionario.id = :id")
    long sumPaginasByFuncionarioId(@Param("id") Long id);
}
```

3.3 EtiquetaRepository.java

```
public interface EtiquetaRepository extends JpaRepository<Etiqueta, Long> {
    Optional<Etiqueta> findByFuncionarioIdAndProdutoId(Long funcionarioId, Long produtoId);
}
```

3.4 AnotacaoRepository.java

```
public interface AnotacaoRepository extends JpaRepository<Anotacao, Long> { }
```

Checkpoint Etapa 3: O código compila sem erros (`mvn compile`).

Etapa 4 — Importar Dados do Prototipo (SQL)

PARTE PRINCIPAL: Os dados devem ser importados via `data.sql` para popular o banco.

4.1 `data.sql` — `src/main/resources/data.sql`

4.1.1 Inserts para `tb_funcionarios`

```
INSERT IGNORE INTO tb_funcionarios (id, nome, email, nome_usuario, imagem, senha, created_at, updated_at) VALUES
('1', 'saepestoque', 'saepestoque@email.com', 'saepestoque', 'saepestoque.png', '123456', '2024-08-14 18:56:33', '2024-08-14 18:56:33'),
('2', 'operador1', 'operador1@email.com', 'operador01', 'operador01.jpg', '123456', '2024-08-14 18:58:07', '2024-08-14 18:58:07'),
('3', 'operador2', 'operador2@email.com', 'operador02', 'operador02.jpg', '123456', '2024-08-14 18:58:21', '2024-08-14 18:58:21'),
('4', 'operador3', 'operador3@email.com', 'operador03', 'operador03.jpg', '123456', '2024-08-14 18:58:35', '2024-08-14 18:58:35');
```

4.1.2 Inserts para `tb_produtos`

```
INSERT IGNORE INTO tb_produtos (id, genero, titulo, numero_paginas, ano_publicacao, created_at, updated_at, funcionario_id) VALUES
(3, 'Higiene', 'Sabonete Liquido', 12.9, 500, '2024-08-14 19:15:11', '2024-08-14 19:15:11', 1),
(4, 'Higiene', 'Shampoo Anticaspa', 22.5, 350, '2024-08-14 19:15:54', '2024-08-14 19:15:54', 2),
(5, 'Higiene', 'Creme Dental', 8.9, 800, '2024-08-14 19:16:09', '2024-08-14 19:16:09', 3),
(6, 'Higiene', 'Desodorante Aerosol', 15.9, 420, '2024-08-14 19:16:24', '2024-08-14 19:16:24', 4),
(7, 'Limpeza', 'Detergente Liquido', 3.5, 1200, '2024-08-14 19:17:30', '2024-08-14 19:17:30', 1),
(8, 'Limpeza', 'Agua Sanitaria', 5.9, 950, '2024-08-14 19:17:47', '2024-08-14 19:17:47', 2),
(9, 'Limpeza', 'Desinfetante Lavanda', 9.9, 680, '2024-08-14 19:18:18', '2024-08-14 19:18:18', 3),
(10, 'Limpeza', 'Limpa Vidros', 7.5, 550, '2024-08-14 19:18:41', '2024-08-14 19:18:41', 4),
(11, 'Alimenticio', 'Arroz 5kg', 22, 300, '2024-08-14 19:20:24', '2024-08-14 19:20:24', 1),
(12, 'Alimenticio', 'Feijao 1kg', 8.5, 450, '2024-08-14 19:20:43', '2024-08-14 19:20:43', 2),
(13, 'Alimenticio', 'Oleo de Soja', 6.9, 600, '2024-08-14 19:26:04', '2024-08-14 19:26:04', 3),
(14, 'Alimenticio', 'Macarrao 500g', 4.5, 750, '2024-08-14 19:26:23', '2024-08-14 19:26:23', 4);
```

IMPORTANTE: `INSERT IGNORE` evita duplicar dados ao reiniciar a aplicacao. Os IDs fixos garantem integridade das chaves estrangeiras.

Checkpoint Etapa 4: Apos subir a aplicacao, `SELECT * FROM tb_funcionarios;` e `SELECT * FROM tb_produtos;` retornam os dados.

Etapa 5 — Criar a API REST (Controller)

5.1 ApiController.java

Login

```
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody LoginRequest request) {
    Optional<Funcionario> funcionario =
        funcionarioRepository.findByEmailAndSenha(request.getEmail(), request.getSenha());
    if (funcionario.isPresent())
        return ResponseEntity.ok(funcionario.get());
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Email ou senha incorreta");
}
```

Perfil — Totais

```
@GetMapping("/perfil/{usuarioId}")
public ResponseEntity<?> getPerfil(@PathVariable Long usuarioId) {
    long total = produtoRepository.countByFuncionarioId(usuarioId);
    long soma = produtoRepository.sumPaginasByFuncionarioId(usuarioId);
    Map<String, Object> response = new HashMap<>();
    response.put("totalAtividades", total);
    response.put("totalCalorias", soma);
    return ResponseEntity.ok(response);
}
```

Listar com Paginação e Filtro

```
@GetMapping("/produtos")
public ResponseEntity<Page<Produto>> getItens(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(required = false) String tipo) {
    Pageable pageable = PageRequest.of(page, 4); // 4 por pagina
    return ResponseEntity.ok(produtoRepository.findByGenero(tipo, pageable));
}
```

Criar

```
@PostMapping("/produtos")
public ResponseEntity<?> criar(@RequestBody CriarRequest request) {
    Funcionario funcionario = funcionarioRepository.findById(request.getUsuarioId()).orElse(null);
    if (funcionario == null)
        return ResponseEntity.badRequest().body("Funcionario nao encontrado");
    Produto item = new Produto();
    item.setTitulo(request.getTitulo());
    item.setAutor(request.getAutor());
    item.setGenero(request.getGenero());
    item.setAnoPublicacao(request.getAnoPublicacao());
    item.setCreatedAt(LocalDateTime.now());
    item.setUpdatedAt(LocalDateTime.now());
    item.setFuncionario(funcionario);
    produtoRepository.save(item);
    return ResponseEntity.ok(item);
}
```

Reacao (Toggle)

```
@PostMapping("/produtos/{id}/etiquetar")
public ResponseEntity<?> reagir(@PathVariable Long id, @RequestBody InteracaoRequest request) {
```



```
Optional<Etiqueta> existe = etiquetaRepository
    .findByFuncionarioIdAndProdutoId(request.getUsuarioId(), id);
if (existe.isPresent()) {
    etiquetaRepository.delete(existe.get());
    return ResponseEntity.ok("Removeu etiqueta");
}
Etiqueta nova = new Etiqueta();
nova.setFuncionario(funcionarioRepository.findById(request.getUsuarioId()).get());
nova.setProduto(produtoRepository.findById(id).get());
etiquetaRepository.save(nova);
return ResponseEntity.ok("Etiquetou");
}
```

Comentario

```
@PostMapping("/produtos/{id}/anotar")
public ResponseEntity<> comentar(@PathVariable Long id, @RequestBody InteracaoRequest request) {
    if (request.getTexto() == null || request.getTexto().length() <= 2)
        return ResponseEntity.badRequest().body("nao e possivel enviar um comentario vazio");
    Anotacao c = new Anotacao();
    c.setFuncionario(funcionarioRepository.findById(request.getUsuarioId()).get());
    c.setProduto(produtoRepository.findById(id).get());
    c.setTexto(request.getTexto());
    anotacaoRepository.save(c);
    return ResponseEntity.ok(c);
}
```

5.2 Mapa de Endpoints

Endpoint	Metodo	Descricao
/api/login	POST	Login do funcionario
/api/perfil/{id}	GET	Totais do funcionario
/api/produtos?page=&categoria=	GET	Listar com filtro
/api/produtos	POST	Cadastrar produto
/api/produtos/{id}/etiquetar	POST	Etiquetar/Remover
/api/produtos/{id}/anotar	POST	Adicionar anotacao

Checkpoint Etapa 5: Teste com Postman: GET `http://localhost:8080/api/produtos?page=0` e POST `http://localhost:8080/api/login` com `{"email": "saepestoque@email.com", "senha": "123456"}` .

Etapa 6 — Estrutura HTML da SPA (Frontend)

PARTE PRINCIPAL: O Spring Boot serve arquivos estaticos da pasta `src/main/resources/static/` . A SPA nunca recarrega: tudo via `fetch()` .

6.1 `index.html`

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SAEPEstoque</title>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="almoxarifado-app">

    <!-- COLUNA ESQUERDA: PERFIL -->
    <aside class="barra-funcionario">
      <div class="identificacao-func">
        
        <h2 id="cracha-func">SAEPEstoque</h2>

        <div class="contadores">
          <div class="caixa-contador">
            <span id="contador-produtos" class="digito-contador">0</span>
            <span class="legenda-contador">Total de Produtos</span>
          </div>
          <div class="caixa-contador">
            <span id="contador-valor" class="digito-contador">0</span>
            <span class="legenda-contador">Valor em Estoque</span>
          </div>
        </div>

        <button id="btn-cadastrar-produto" class="btn-cadastro" disabled>
           Produto
        </button>
      </div>

      <footer class="rodape-func">
        <p>SAEPEstoque</p>
        <div class="icones-sociais">
          
          
          
        </div>
        <p class="direitos-autorais">Copyright - 2024</p>
      </footer>
    </aside>

    <!-- COLUNA CENTRAL: CONTEUDO -->
    <main class="painel-estoque">
      <header class="cabecalho-estoque">
        <button id="btn-credencial" class="btn-credenciar">Login</button>
      </header>

      <div class="secao-categorias">
        Higiene
        Limpeza
        Alimenticio
      </div>

      <div id="lista-atividades" class="grade-produtos"></div>

      <div class="botoes-paginacao">
        <button id="btn-pagina-anterior" class="botao-indice">Anterior</button>
```

```

        <div id="indice-paginas" class="numeros-pagina"></div>
        <button id="btn-pagina-seguite" class="botao-indice">Proxima</button>
    </div>
</main>
</div>

<!-- MODAL LOGIN -->
<div id="sobreposicao-login" class="sobreposicao">
    <div class="caixa-dialogo">
        <div class="topo-caixa">
            <h2>Login</h2>
            
        </div>
        <form id="form-acesso-estoque">
            <div class="grupo-entrada">
                <label for="credencial-email">Email</label>
                <input type="email" id="credencial-email">
            </div>
            <div class="grupo-entrada">
                <label for="credencial-senha">Senha</label>
                <input type="password" id="credencial-senha">
            </div>
            <div class="painel-confirmacao">
                <button type="button" id="btn-ignorar-login" class="btn-ignorar">Cancelar</button>
                <button type="submit" class="btn-processar">Login</button>
            </div>
        </form>
    </div>
</div>

```

```

<!-- MODAL CRIAR -->
<div id="sobreposicao-produto" class="sobreposicao">
    <div class="caixa-dialogo">
        <div class="topo-caixa">
            <h2>Crie seu registro</h2>
            
        </div>
        <form id="form-produto">
            <div class="fila-entradas">
                <div class="grupo-entrada metade-largura">
                    <label for="selecao-categoria">Tipo</label>
                    <select id="selecao-categoria" required>
                        <option value="">Selecione...</option>

```

Higiene

Limpeza

Alimenticio

```

            </select>
        </div>
        <div class="grupo-entrada metade-largura">
            <label for="quantidade-produto">Numero de Paginas</label>
            <input type="number" id="quantidade-produto" placeholder="Ex: 300" required>
        </div>
    </div>
    <div class="fila-entradas">
        <div class="grupo-entrada metade-largura">
            <label for="nome-produto">Autor</label>
            <input type="text" id="nome-produto" placeholder="Ex: Machado de Assis">
        </div>
        <div class="grupo-entrada metade-largura">
            <label for="preco-produto">Ano de Publicacao</label>
            <input type="number" id="preco-produto" placeholder="Ex: 1899" required>
        </div>
    </div>
    <div class="painel-confirmacao">
        <button type="submit" class="btn-processar">Criar Produto</button>
    </div>
</form>
</div>
</div>

```

```

    <script src="app.js"></script>
</body>
</html>

```

6.2 Ativos estaticos para `static/assets/`

```
src/main/resources/static/assets/  
├─ logo_saepestoque/ Logo.png  
├─ icones/ (coracao.svg, CoracaoVermelho.svg, comentario.svg, send.svg, x.svg, ...)  
├─ imagens_perfil/ (leitor01.jpg, leitor02.jpg, leitor03.jpg)  
└─ ...
```

Checkpoint Etapa 6: Acessar `http://localhost:8080` mostra o layout (sidebar escura + area principal), mesmo sem dados.

Etapa 7 — Folha de Estilos CSS

PARTE PRINCIPAL: Estrutura organizada com cores: #6c3483 (fundo sidebar), #4a235a (destaque), #FFFFFF (fontes).

7.1 style.css

```
/* =====  
SAPEstoque — Folha de Estilos  
Fonte: Inter | Cores: #6c3483 #FFFFFF #4a235a #e74c3c  
===== */  
  
/* --- RESET --- */  
* { margin: 0; padding: 0; box-sizing: border-box; font-family: 'Inter', sans-serif; }  
body { background-color: #f5f5f5; }  
  
/* --- LAYOUT GERAL --- */  
.almoxarifado-app { display: flex; min-height: 100vh; }  
  
/* --- BARRA LATERAL — fundo #6c3483 --- */  
.barra-funcionario {  
  width: 300px;  
  background-color: #6c3483;  
  color: #FFFFFF;  
  display: flex;  
  flex-direction: column;  
  justify-content: space-between;  
}  
  
.identificacao-func { padding: 40px 20px; text-align: center; }  
  
.foto-func {  
  width: 120px; height: 120px; border-radius: 50%;  
  margin-bottom: 20px; object-fit: cover;  
  border: 3px solid #4a235a;  
}  
  
/* --- ESTATISTICAS --- */  
.contadores { display: flex; justify-content: space-around; margin: 30px 0; }  
.caixa-contador { display: flex; flex-direction: column; }  
.digito-contador { font-size: 24px; font-weight: bold; }  
.legenda-contador { font-size: 12px; color: #cccccc; }  
  
/* --- BOTAO CRIAR --- */  
.btn-cadastro {  
  width: 100%; padding: 15px; background: transparent;  
  border: 1px solid #FFFFFF; color: #FFFFFF;  
  cursor: pointer; display: flex; align-items: center; justify-content: center;  
  gap: 10px; font-size: 16px;  
  transition: background-color 0.3s, border-color 0.3s;  
}  
.btn-cadastro:hover:not(:disabled) { background-color: #4a235a; border-color: #4a235a; }  
.btn-cadastro:disabled { opacity: 0.5; cursor: not-allowed; }  
  
/* --- RODAPE --- */  
.rodape-func { text-align: center; padding: 20px; border-top: 1px solid #444; font-size: 12px; color: #999; }  
  
/* --- CONTEUDO PRINCIPAL --- */  
.painel-estoque { flex: 1; display: flex; flex-direction: column; background-color: #f9f9f9; }  
.cabecalho-estoque { display: flex; justify-content: flex-end; padding: 20px; }  
  
/* --- BOTÕES AUTENTICACAO --- */  
.btn-credenciar {  
  padding: 10px 30px; background-color: #FFFFFF;  
  color: #6c3483; border: 1px solid #6c3483;  
  border-radius: 4px; font-weight: bold; cursor: pointer;  
}  
.btn-descredenciar {  
  padding: 10px 30px; background-color: #6c3483;  
  color: #FFFFFF; border: none;  
  border-radius: 4px; font-weight: bold; cursor: pointer;
```

```

}

/* --- FILTROS - fundo #6c3483 --- */
.secao-categorias { display: flex; background-color: #6c3483; }
.btn-categoria-estoque {
  flex: 1; padding: 15px; background: none; border: none;
  border-right: 1px solid #444; color: #FFFFFF;
  font-size: 16px; cursor: pointer;
}
.btn-categoria-estoque:last-child { border-right: none; }
.btn-categoria-estoque:hover, .btn-categoria-estoque.ativo { background-color: #4a235a; }

/* --- LISTA DE ITENS --- */
.grade-produtos { padding: 20px; display: flex; flex-direction: column; gap: 15px; }
.cartela-produto { background: #FFFFFF; border: 1px solid #ddd; border-radius: 8px; padding: 15px; }
.rotulo-produto { width: 100%; margin-bottom: 10px; text-align: center; font-weight: bold; }

/* --- INTERACOES --- */
.controles-produto { display: flex; gap: 20px; align-items: center; }
.botao-control { display: flex; align-items: center; gap: 5px; cursor: pointer; }
.botao-control img { width: 24px; }

/* --- CAIXA DE COMENTARIO --- */
.bloco-anotacao { width: 100%; margin-top: 15px; display: none; align-items: center; gap: 10px; }
.campo-anotacao { flex: 1; padding: 10px; border: 1px solid #ccc; border-radius: 4px; }

/* --- PAGINACAO --- */
.botoes-paginacao { display: flex; justify-content: center; align-items: center; gap: 10px; padding: 20px; margin-top: auto; }
.botao-indice { padding: 5px 15px; border: 1px solid #ccc; background: #FFFFFF; cursor: pointer; }
.botao-indice:disabled { opacity: 0.5; cursor: not-allowed; }

/* --- MODAIS --- */
.sobreposicao { display: none; position: fixed; top: 0; left: 0; width: 100%; height: 100%;
  background: rgba(0,0,0,0.5); justify-content: center; align-items: center; }
.sobreposicao.ativo { display: flex; }
.caixa-dialogo { width: 400px; padding: 30px; background: #FFFFFF; border-radius: 8px; }
.topo-caixa { display: flex; justify-content: space-between; align-items: center; margin-bottom: 20px; }
.icone-x { width: 20px; cursor: pointer; }

/* --- FORMULARIOS --- */
.grupo-entrada { display: flex; flex-direction: column; margin-bottom: 15px; }
.fila-entradas { display: flex; gap: 20px; }
.metade-largura { flex: 1; }
.grupo-entrada label { margin-bottom: 5px; font-weight: 600; }
.grupo-entrada input, .grupo-entrada select {
  padding: 10px; border: 1px solid #ccc; border-radius: 4px;
}
.grupo-entrada input.erro, .grupo-entrada select.erro { border-color: #e74c3c; }

/* --- ACOES MODAIS --- */
.painel-confirmacao { display: flex; justify-content: flex-end; gap: 10px; margin-top: 20px; }
.btn-ignorar { padding: 10px 20px; background: none; border: 1px solid #6c3483;
  border-radius: 4px; color: #6c3483; cursor: pointer; }
.btn-processar { padding: 10px 20px; background: #6c3483; border: none;
  border-radius: 4px; color: #FFFFFF; cursor: pointer; }

```

Checkpoint Etapa 7: O visual corresponde as cores: sidebar #6c3483 , filtros #6c3483 / #FFFFFF , paginacao ativa #4a235a .

Etapas 8 — JavaScript da SPA (Foco: Modal de Login)

PARTE PRINCIPAL: O `app.js` faz todas as chamadas `fetch` a API e atualiza a tela sem recarregar (regra SPA). Abaixo, o código completo com foco no modal de login.

8.1 `app.js`

```
const API_URL = '/api';
let usuarioLogado = null;
let paginaAtual = 0;
let filtroAtual = null;
let totalPaginas = 1;

document.addEventListener('DOMContentLoaded', () => {
  const armazenado = localStorage.getItem('usuario');
  if (armazenado) {
    usuarioLogado = JSON.parse(armazenado);
    atualizarUIUsuarioLogado();
    buscarEstatisticasPerfil();
  }
  carregarItens();

  // Event Listeners
  document.getElementById('btn-credencial').addEventListener('click', tratarCliqueAutenticacao);
  document.getElementById('x-login').addEventListener('click', () => alternarModal('sobreposicao-login', false));
  document.getElementById('btn-ignorar-login').addEventListener('click', () => alternarModal('sobreposicao-login', false));
  document.getElementById('form-acesso-estoque').addEventListener('submit', tratarLogin);

  document.getElementById('btn-cadastrar-produto').addEventListener('click', () => alternarModal('sobreposicao-produto', true));
  document.getElementById('x-produto').addEventListener('click', () => alternarModal('sobreposicao-produto', false));
  document.getElementById('form-produto').addEventListener('submit', tratarCriar);

  document.getElementById('btn-pagina-anterior').addEventListener('click', () => {
    if (paginaAtual > 0) { paginaAtual--; carregarItens(); }
  });
  document.getElementById('btn-pagina-seguinte').addEventListener('click', () => {
    if (paginaAtual < totalPaginas - 1) { paginaAtual++; carregarItens(); }
  });

  // Filtros bloqueados se não logado
  document.querySelectorAll('.btn-categoria-estoque').forEach(btn => {
    btn.addEventListener('click', (e) => {
      if (!usuarioLogado) { alternarModal('sobreposicao-login', true); return; }
      const tipo = e.target.getAttribute('data-tipo');
      if (filtroAtual === tipo) { filtroAtual = null; e.target.classList.remove('ativo'); }
      else { filtroAtual = tipo;
        document.querySelectorAll('.btn-categoria-estoque').forEach(b => b.classList.remove('ativo'));
        e.target.classList.add('ativo'); }
      paginaAtual = 0;
      carregarItens();
    });
  });

  // ===== FUNCAO ALTERNAR MODAL =====
  function alternarModal(idModal, mostrar) {
    document.getElementById(idModal).classList.toggle('ativo', mostrar);
  }

  // ===== FUNCAO TRATAR AUTENTICACAO =====
  function tratarCliqueAutenticacao() {
    if (usuarioLogado) {
      usuarioLogado = null;
      localStorage.removeItem('usuario');
      window.location.reload();
    } else {
      alternarModal('sobreposicao-login', true);
    }
  }
}
```

```

// ===== FUNCAO LOGIN (MODAL) =====
async function tratarLogin(e) {
  e.preventDefault();
  const campoEmail = document.getElementById('credencial-email');
  const campoSenha = document.getElementById('credencial-senha');
  campoEmail.classList.remove('erro');
  campoSenha.classList.remove('erro');

  // Validacao: campos obrigatorios
  if (!campoEmail.value || !campoSenha.value) {
    if (!campoEmail.value) campoEmail.classList.add('erro');
    if (!campoSenha.value) campoSenha.classList.add('erro');
    alert('email ou senha obrigatorio');
    return;
  }

  try {
    const res = await fetch(`${API_URL}/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email: campoEmail.value, senha: campoSenha.value })
    });

    if (res.ok) {
      // Login bem-sucedido
      usuarioLogado = await res.json();
      localStorage.setItem('usuario', JSON.stringify(usuarioLogado));
      atualizarUIUsuarioLogado();
      alternarModal('sobreposicao-login', false);
      buscarEstatisticasPerfil();
      carregarItens();
    } else {
      // Credenciais invalidas
      campoEmail.classList.add('erro');
      campoSenha.classList.add('erro');
      alert('email ou senha incorreta');
    }
  } catch (err) {
    console.error(err);
  }
}

// ===== ATUALIZAR UI APOS LOGIN =====
function atualizarUIUsuarioLogado() {
  const botao = document.getElementById('btn-credencial');
  botao.textContent = 'Logout';
  botao.classList.replace('btn-credenciar', 'btn-descredenciar');
  document.getElementById('cracha-func').textContent = usuarioLogado.nomeUsuario;
  document.getElementById('avatar-func').src = `assets/imagens_perfil/${usuarioLogado.imagem}`;
  document.getElementById('btn-cadastrar-produto').disabled = false;
}

// ===== BUSCAR ESTATISTICAS DO PERFIL =====
async function buscarEstatisticasPerfil() {
  if (!usuarioLogado) return;
  const res = await fetch(`${API_URL}/perfil/${usuarioLogado.id}`);
  const stats = await res.json();
  document.getElementById('contador-produtos').textContent = stats.totalAtividades;
  document.getElementById('contador-valor').textContent = stats.totalCalorias;
}

// ===== CARREGAR ITENS COM PAGINACAO =====
async function carregarItens() {
  let url = `${API_URL}/produtos?page=${paginaAtual}`;
  if (filtroAtual) url += `&tipo=${filtroAtual}`;
  try {
    const res = await fetch(url);
    const data = await res.json();
    totalPaginas = data.totalPages;
    renderizarItens(data.content);
    atualizarPaginacao();
  } catch (err) { console.error(err); }
}

// ===== RENDERIZAR ITENS =====
function renderizarItens(itens) {
  const lista = document.getElementById('lista-atividades');
  lista.innerHTML = '';
  itens.forEach(item => {
    const d = new Date(item.createdAt);

```



```

const dataFormatada =
` ${String(d.getHours()).padStart(2, '0')}:${String(d.getMinutes()).padStart(2, '0')} - ` +
` ${String(d.getDate()).padStart(2, '0')}/${String(d.getMonth() + 1).padStart(2, '0')}/${d.getFullYear()}`;

const jaReagiu = usuarioLogado && item.etiquetas.some(r => r.funcionario.id === usuarioLogado.id);
const icone = jaReagiu ? 'CoracaoVermelho.svg' : 'coracao.svg';

const cartao = document.createElement('div');
cartao.className = 'cartela-produto';
cartao.innerHTML = `
  <div class="rotulo-produto">
    <span style="text-transform:capitalize">${item.genero}</span>
    <span style="float:right;font-size:12px;font-weight:normal">${dataFormatada}</span>
  </div>
  <div class="especificacao-produto">
    
    <div>
      <strong>${item.funcionario.nomeUsuario}</strong><br>
      <strong>${item.titulo}</strong><br>
      ${item.numeroPaginas} páginas | Ano: ${item.anoPublicacao}
    </div>
    <div class="controles-produto">
      <div class="botao-controle" onclick="tratarReacao(${item.id})">
        
        <span>${item.etiquetas.length}</span>
      </div>
      <div class="botao-controle" onclick="alternarCaixaComentario(${item.id})">
        
        <span>${item.anotacoes.length}</span>
      </div>
    </div>
    <div id="caixa-comentario-${item.id}" class="bloco-anotacao">
      <input type="text" id="entrada-comentario-${item.id}" class="campo-anotacao" placeholder="Escrever um comentario...">
      
    </div>
  </div>`;
  lista.appendChild(cartao);
});
}

// ===== ATUALIZAR PAGINACAO =====
function atualizarPaginacao() {
  document.getElementById('btn-pagina-anterior').disabled = paginaAtual === 0;
  document.getElementById('btn-pagina-seguinte').disabled = paginaAtual >= totalPaginas - 1;
  const numeros = document.getElementById('indice-paginas');
  numeros.innerHTML = '';
  const botao = document.createElement('button');
  botao.textContent = paginaAtual + 1;
  botao.className = 'ativo';
  numeros.appendChild(botao);
}

// ===== REACAO (CURTIR/DESCURTIR) =====
async function tratarReacao(id) {
  if (!usuarioLogado) { alternarModal('sobreposicao-login', true); return; }
  await fetch(`${API_URL}/produtos/${id}/etiquetar`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ usuarioId: usuarioLogado.id })
  });
  carregarItens();
}

// ===== ALTERNAR CAIXA DE COMENTARIO =====
function alternarCaixaComentario(id) {
  if (!usuarioLogado) { alternarModal('sobreposicao-login', true); return; }
  const caixa = document.getElementById(`caixa-comentario-${id}`);
  caixa.style.display = caixa.style.display === 'flex' ? 'none' : 'flex';
}

// ===== ENVIAR COMENTARIO =====
async function enviarComentario(id) {
  const entrada = document.getElementById(`entrada-comentario-${id}`);
  const texto = entrada.value.trim();
  if (texto.length <= 2) {
    alert('nao e possivel enviar um comentario vazio');
    return;
  }
  await fetch(`${API_URL}/produtos/${id}/anotar`, {

```

```

    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ usuarioId: usuarioLogado.id, texto: texto })
  });
  carregarItens();
}

// ===== CRIAR NOVO ITEM =====
async function tratarCriar(e) {
  e.preventDefault();
  const tipo = document.getElementById('selecao-categoria');
  const distancia = document.getElementById('preco-produto');
  const duracao = document.getElementById('quantidade-produto');
  [tipo, duracao].forEach(el => el.classList.remove('erro'));

  if (!tipo.value || !duracao.value) {
    if (!tipo.value) tipo.classList.add('erro');
    if (!duracao.value) duracao.classList.add('erro');
    alert('Campo obrigatorio');
    return;
  }

  await fetch(`${API_URL}/produtos`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      usuarioId: usuarioLogado.id,
      genero: tipo.value,
      anoPublicacao: parseInt(distancia.value) || 0,
      numeroPaginas: parseInt(duracao.value),
      titulo: document.getElementById('nome-produto').value || 'Sem titulo',
      autor: document.getElementById('nome-produto').value || 'Desconhecido'
    })
  });
  alternarModal('sobreposicao-produto', false);
  paginaAtual = 0;
  carregarItens();
  buscarEstatisticasPerfil();
}

```

Checkpoint Etapa 8: Sem login: filtros/reacao/comentario abrem o modal de login. Logando com `saepestoque@email.com` /

`123456` : a sidebar mostra o usuario, os totais carregam e as interacoes sao liberadas.

Etapa 9 — Checklist Final de Requisitos

#	Requisito	Status
1	Body dividido em colunas (sidebar + main)	[]
2	Sidebar com fundo #6c3483 , fonte #FFFFFF	[]
3	Totais de Total de Produtos e Valor em Estoque vindos do banco	[]
4	Botao "Produto" desabilitado sem login, cor #FFFFFF	[]
5	Rodape com icones e "Copyright-2024"	[]
6	Botao Login no canto superior direito (bg #FFFFFF)	[]
7	Filtros Higiene/Limpeza/Alimenticio centralizados, fundo #6c3483	[]
8	Cada item: titulo centralizado, dados do funcionario, data HH:MM - DD/MM/YY	[]
9	Icones coracao e comentario com contagem	[]
10	Paginacao limita 4 itens; pagina ativa com #4a235a	[]
11	Filtros, paginacao, reacao e comentario bloqueados sem login (abre modal)	[]
12	Modal login: campos email/senha com labels	[]
13	Botoes "Cancelar" (sem bg, borda #6c3483) e "Login" (bg #6c3483)	[]
14	Campo vazio: borda vermelha + mensagem de erro	[]
15	Credencial errada: borda vermelha + "email ou senha incorreta"	[]
16	Login correto: perfil atualiza com dados do funcionario	[]
17	Logout retorna a tela inicial	[]
18	Reacao toggle: clica, conta +1; clica de novo, -1	[]
19	Comentario > 2 caracteres; senao avisa "comentario vazio"	[]
20	Comentario e reacao persistidos no banco	[]
21	Cadastro de novo produto: campos obrigatorios validados	[]
22	Novo produto aparece no topo da lista	[]

Etapa 10 — Rodar e Testar End-to-End

10.1 Fluxo de Teste Completo

1. Pagina carrega com feed de 4 itens (paginacao) e sidebar do sistema.
2. Clicar em filtro/reacao sem login: abre modal de login.
3. Logar com `saepestoque@email.com` / `123456` : perfil muda, totais atualizam.
4. Reagir a um item: icone fica vermelho, contador +1; clicar de novo: -1.
5. Comentar com > 2 caracteres: comentario persiste.
6. Clicar em "Produto": modal de cadastro; preencher e salvar: aparece no topo.
7. Navegar entre paginas com Proxima/Anterior.
8. Logout: volta a tela inicial.

10.2 Estrutura Final de Arquivos

```
src/main/java/com/saep/saepestoque/
├── SAEPEstoqueApplication.java
├── model/
│   ├── Funcionario.java
│   ├── Produto.java
│   ├── Etiqueta.java
│   └── Anotacao.java
├── repository/
│   ├── FuncionarioRepository.java
│   ├── ProdutoRepository.java
│   ├── EtiquetaRepository.java
│   └── AnotacaoRepository.java
└── controller/
    └── ApiController.java

src/main/resources/
├── application.properties
├── data.sql
├── static/
│   ├── index.html
│   ├── style.css
│   ├── app.js
│   └── assets/ (icones, imagens_perfil, logo_*, ...)
```

10.3 Comandos para Executar

```
# 1. Garanta que o MySQL 8 esta rodando localmente (porta 3306)
# 2. Rodar a aplicacao
mvn spring-boot:run

# 3. Abrir no navegador
http://localhost:8080
```